

SIMATS ENGINEERING

ASSESSMENT TOOL 1

CONCEPT MAPPING

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

Course Outcome Covered: CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 60%

Course Faculty: Dr Madhan V

Concept Maps Submitted

Q. No.	Concept Map Topic
1	Software Development Life Cycle (SDLC)
2	Object-Oriented Principles
5	Unified Modeling Language (UML) Diagram Types

Student Details

Particular	Details
Name of the Student	M Nishanth
Register Number	192511154
Department / Year	B. Tech / 2 nd Year
Date of Submission	05-10-2026

Annexure A: Code of Conduct

CONCEPT MAPPING

I _____**M Nishanth (192511154)**____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____**M Nishanth**_____

Date: _____**05-10-2026**_____

1. Introduction and Objective

A concept map is a graphical tool that organises knowledge by placing concepts in boxes and joining them with labelled lines. Each pair of concepts and its linking phrase form a short proposition, and the map is usually arranged from general concepts at the top to specific concepts below (Novak & Cañas, 2008). Concept maps are therefore well suited to object-oriented analysis and design, where many related ideas such as life-cycle phases, principles and modelling notations must be understood together rather than as isolated definitions.

The objective of this submission is to construct three complete concept maps from Assessment Tool 1 (Concept Mapping) of the course Object Oriented Analysis and Design, in line with Course Outcome CO1. For each selected question the missing concepts are identified, the map is drawn, every concept and link is explained, the importance of the concept is analysed, and the answer is supported with standard literature.

1.1 Questions Selected

Q. No.	Topic	Missing Concepts Identified	Marks
1	Software Development Life Cycle	Design, Deployment	5
2	Object-Oriented Principles	Encapsulation, Polymorphism	5
5	UML Diagram Types	Object Diagram, Communication Diagram	5

These three questions were chosen because they form a connected chain. The life cycle (Question 1) describes when software is analysed and designed, the object-oriented principles (Question 2) describe how a design is structured, and UML (Question 5) describes how that structure is documented and communicated. Section 5 shows these links explicitly.

1.2 Method Followed

- **Identify concepts:** the given nodes were read and the missing nodes were deduced from their position in the sequence or hierarchy.
- **Arrange hierarchically:** the general concept was placed at the top, followed by the main branches, and then supporting details.

- **Label every link:** each arrow carries a linking phrase so that the relationship can be read as a sentence.
- **Explain and analyse:** every answer is followed by an explanation, an analysis of benefits, and literature support.

1.3 How to Read the Concept Maps

In every figure, **blue boxes** are the concepts given in the question, **yellow boxes** are the missing concepts supplied as the answer, and **grey boxes** are supporting concepts added to complete the map. The dark blue box at the top is the root concept. *Italic text on an arrow is the linking phrase.*

2. Concept Map 1: Software Development Life Cycle (Question 1)

2.1 Given

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

2.2 Concept Map

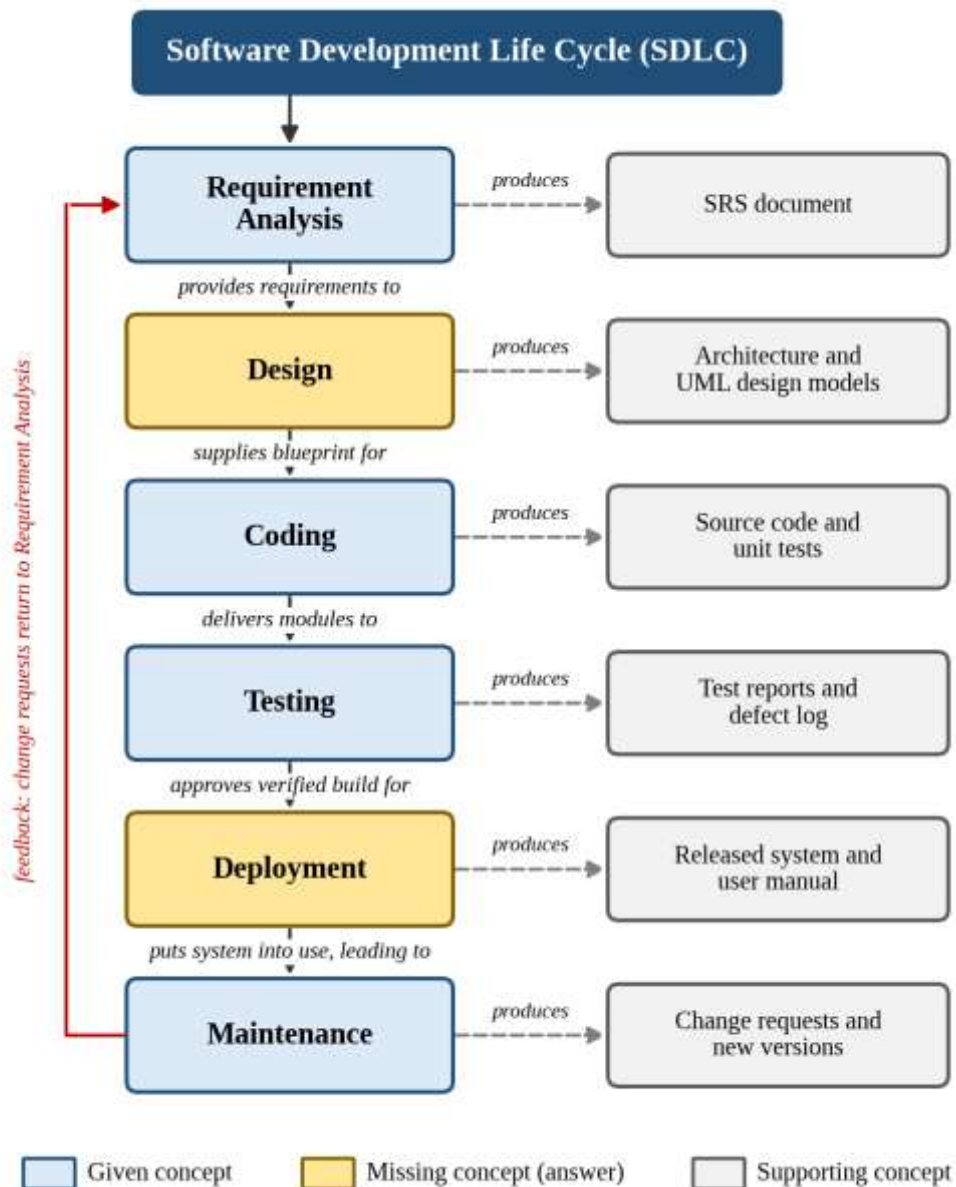


Figure 1: Concept map of the Software Development Life Cycle

2.3 Answer

The missing concepts are **Design** (between Requirement Analysis and Coding) and **Deployment** (between Testing and Maintenance). The completed sequence is:

Requirement Analysis → Design → Coding → Testing → Deployment → Maintenance

2.4 Concept Identification

Concept	Meaning	Main Output
Requirement Analysis	Gathering, analysing and documenting what the users and stakeholders need from the system.	Software Requirements Specification (SRS)
Design	Converting the requirements into a system architecture, modules, interfaces, data structures and class models.	Design document with architecture and UML models
Coding	Writing the source code that implements the design.	Source code and unit tests
Testing	Verifying that the code meets the requirements and finding defects.	Test reports and defect log
Deployment	Releasing the tested software to end users by installation, configuration and training.	Released system and user manual
Maintenance	Correcting faults and updating the software after release (corrective, adaptive, perfective and preventive changes).	Change requests and new versions

2.5 Linkages Between Concepts

From	Linking Phrase	To	Reason for the Link
Requirement Analysis	provides requirements to	Design	Design cannot start until needs are agreed.
Design	supplies blueprint for	Coding	Programmers implement the approved structure.
Coding	delivers modules to	Testing	Only implemented code can be tested.

From	Linking Phrase	To	Reason for the Link
Testing	approves verified build for	Deployment	Software is released only after it passes tests.
Deployment	puts system into use, leading to	Maintenance	Faults and new needs appear once users work with the system.
Maintenance	returns change requests to	Requirement Analysis	Feedback loop that starts the next cycle.

2.6 Explanation

Design converts requirements into system architecture. The requirements describe what the system must do, but not how it will be built. During design the team decides the overall architecture, divides the system into modules, defines the interfaces between modules, plans the database, and, in an object-oriented project, identifies the classes and their relationships. Design must come after Requirement Analysis because it needs the approved requirements as input, and before Coding because programmers need a blueprint to follow. Without it, different programmers would make different structural decisions and the system would become inconsistent.

Deployment releases the software to end users. After testing, the build is packaged, installed in the real working environment, configured, and handed over with documentation and user training. Deployment must come after Testing, because releasing untested software exposes users to defects. It must come before Maintenance, because maintenance only becomes meaningful once real users are using the software and reporting faults or asking for changes.

2.7 Analysis

- **Ensures systematic software development:** each phase has a clear input, output and review point, so progress can be planned, measured and controlled.
- **Reduces project risks:** mistakes found in requirements or design are much cheaper to correct than mistakes found after release (Boehm, 1981; Pressman & Maxim, 2019), so ordering the phases logically lowers cost and schedule risk.
- **Improves software quality and maintainability:** documented requirements, design and test records allow later developers to understand and safely change the system.

A limitation of the strictly sequential form is that it assumes stable requirements. For this reason the feedback arrow from Maintenance back to Requirement Analysis is shown in the map, and iterative models such as incremental and agile development repeat these same phases in short cycles (Sommerville, 2016).

2.8 Support from Literature

The phase sequence in this map follows the sequential development flow described by Royce (1970), who also warned that it carries risk unless there is feedback between phases. Sommerville (2016) and Pressman and Maxim (2019) present the same core activities of requirements, design, implementation, verification, deployment and evolution as the basis of every software process model, which confirms that Design and Deployment are essential, not optional, nodes in this map.

3. Concept Map 2: Object-Oriented Principles (Question 2)

3.1 Given

Object-Oriented Principles → Abstraction → _____ → Inheritance → _____

3.2 Concept Map

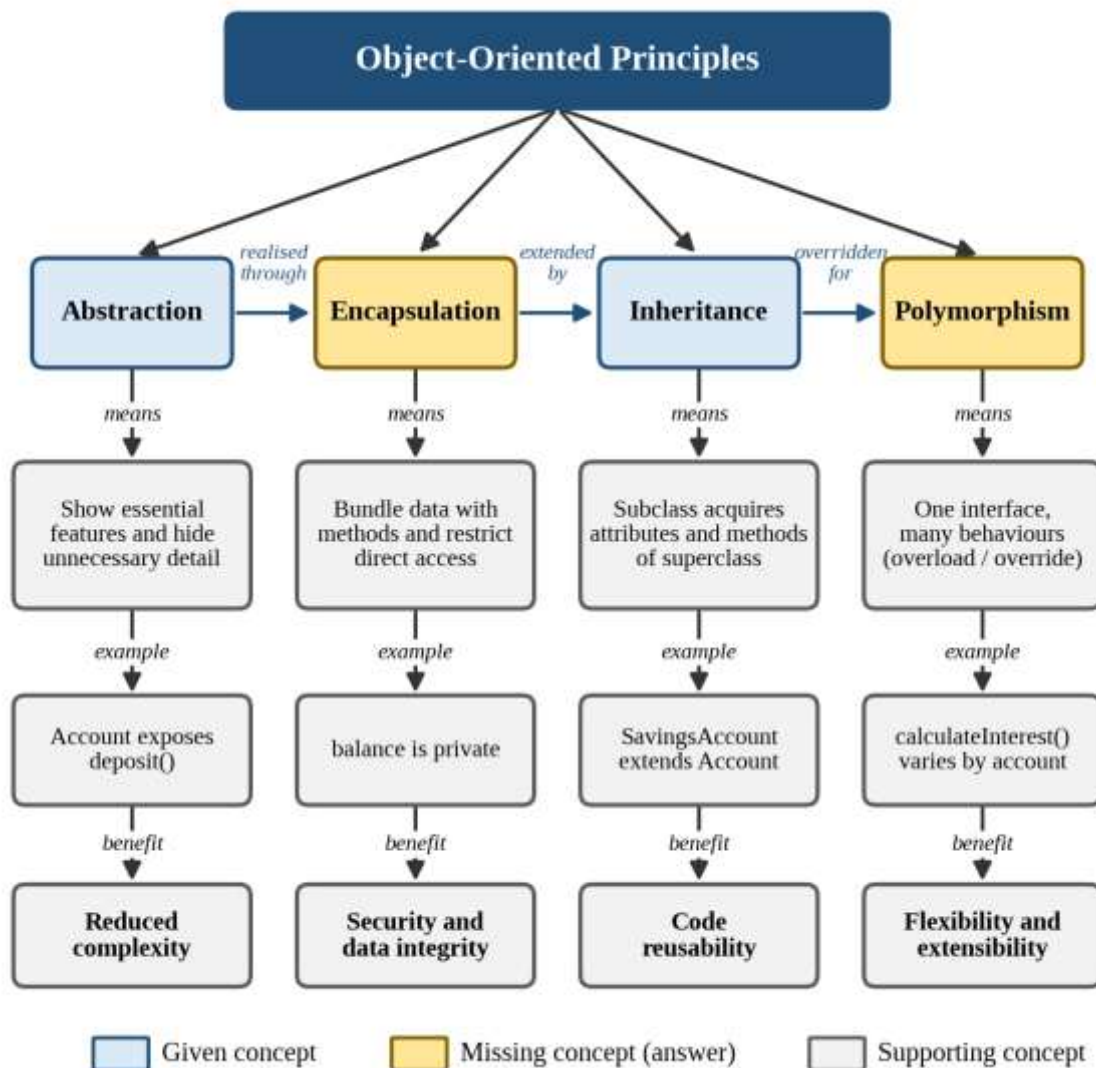


Figure 2: Concept map of the Object-Oriented Principles

3.3 Answer

The missing concepts are **Encapsulation** (between Abstraction and Inheritance) and **Polymorphism** (after Inheritance). The four pillars of object orientation are therefore Abstraction, Encapsulation, Inheritance and Polymorphism.

3.4 Concept Identification

Concept	Meaning	Example (Bank System)
Abstraction	Showing only the essential features of an entity and hiding unnecessary detail.	An Account class exposes deposit() and withdraw() without showing how they work internally.
Encapsulation	Bundling data and the methods that act on it inside a class, and restricting direct access to the data.	The balance attribute is private and can change only through deposit() and withdraw().
Inheritance	A subclass acquires the attributes and methods of a superclass, modelling an "is-a" relationship.	SavingsAccount and CurrentAccount extend Account.
Polymorphism	One interface that supports multiple behaviours, through method overloading (compile time) and overriding (run time).	calculateInterest() gives a different result for each account type.

3.5 Linkages Between Concepts

From	Linking Phrase	To	Reason for the Link
Object-Oriented Principles	is built on	All four principles	These four ideas together define the object-oriented approach.
Abstraction	is realised through	Encapsulation	Abstraction decides what to show; encapsulation enforces it by hiding the rest.
Encapsulation	lets well-defined classes be extended by	Inheritance	Classes with protected internals can be safely reused as superclasses.
Inheritance	enables overriding, which gives	Polymorphism	A subclass can redefine an inherited method, so one call behaves differently.

3.6 Explanation

Encapsulation protects data from unauthorized access. By declaring attributes private and allowing access only through public methods, a class controls how its data can change. This keeps the object in a valid state and hides how the data is stored, an idea that originates from the information-hiding principle of Parnas (1972). It follows Abstraction in the chain because abstraction decides which features are visible, and encapsulation is the mechanism that enforces that decision in code.

Polymorphism allows one interface to support multiple behaviours. A single call such as `calculateInterest()` on an `Account` reference produces the correct result for a savings or current account, because the method is resolved according to the actual object at run time. It follows Inheritance because overriding an inherited method is the main way in which subclasses supply their own behaviour behind a shared interface.

3.7 Analysis

- **Improves security and flexibility:** encapsulation prevents accidental misuse of data, and polymorphism lets new types be added without changing the code that uses the common interface.
- **Enhances code reusability:** inheritance lets common attributes and methods be written once in a superclass and reused by every subclass (Meyer, 1997).
- **Supports efficient software design:** abstraction reduces complexity, so designers can work at the level of the problem domain instead of implementation detail.

3.8 Support from Literature

Booch et al. (2007) identify abstraction and encapsulation as complementary ideas, with abstraction describing the outside view of an object and encapsulation hiding the inside view. Cardelli and Wegner (1985) classify polymorphism into universal forms (parametric and inclusion) and ad hoc forms (overloading and coercion), which matches the overloading and overriding forms listed in the map. Meyer (1997) explains how inheritance and polymorphism support reuse and extension, which is why Inheritance is linked to Polymorphism in the chain.

4. Concept Map 3: UML Diagram Types (Question 5)

4.1 Given

UML → Structural Diagrams → _____ | Behavioral Diagrams → Activity Diagram |
Interaction Diagrams → _____

4.2 Concept Map

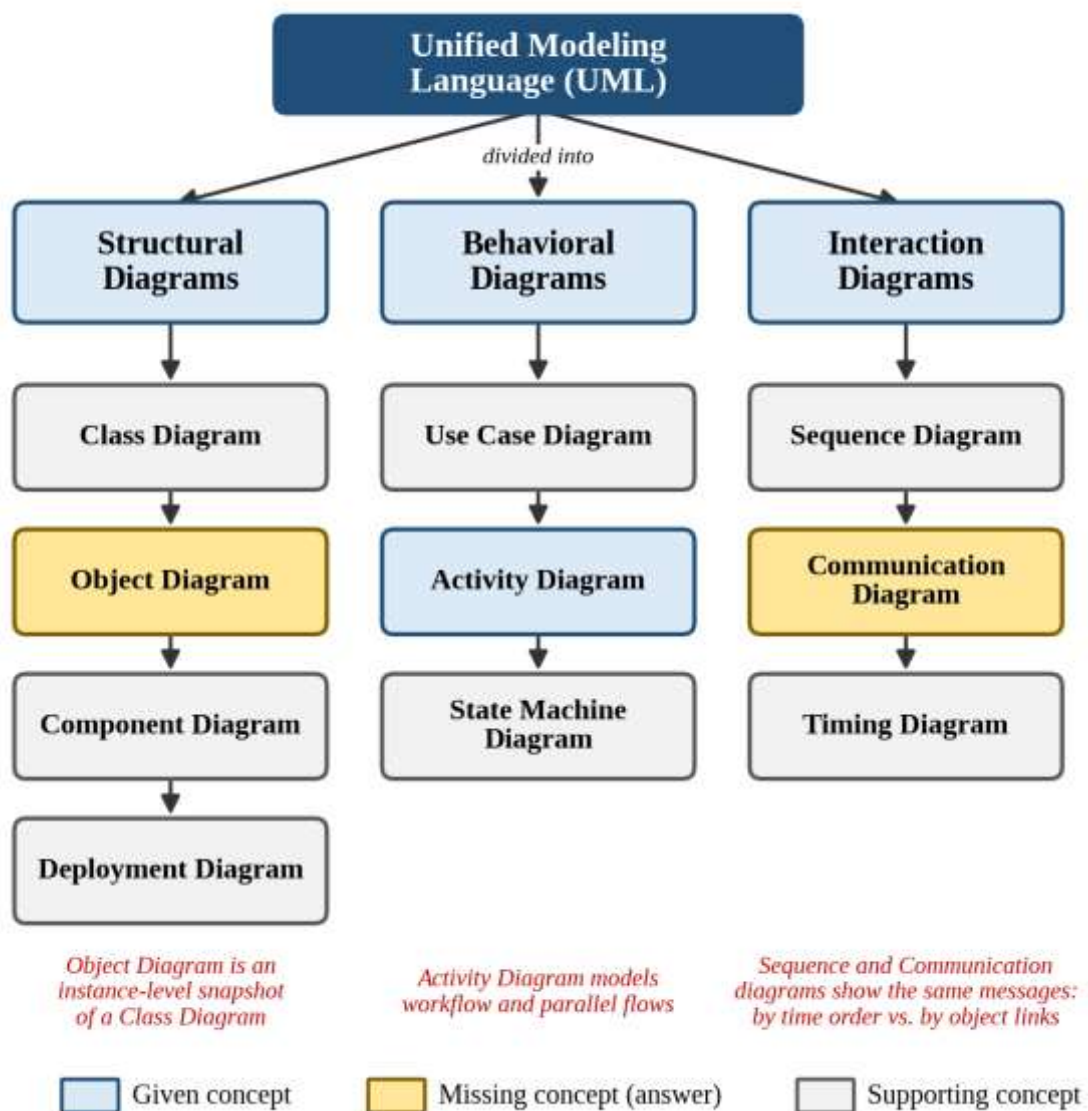


Figure 3: Concept map of the UML diagram types

4.3 Answer

The missing concepts are the **Object Diagram** under Structural Diagrams and the **Communication Diagram** under Interaction Diagrams.

4.4 Concept Identification

Concept	Meaning
UML	The Unified Modeling Language, a standard visual notation for specifying, visualising, constructing and documenting software systems (Object Management Group, 2017).
Structural Diagrams	Show the static parts of a system and their relationships, independent of time. Examples: class, object, component and deployment diagrams.
Object Diagram	Shows a snapshot of object instances, with their attribute values and links, at one moment in time.
Behavioral Diagrams	Show how the system behaves over time. Examples: use case, activity and state machine diagrams.
Activity Diagram	Models a workflow as actions, decisions and parallel flows.
Interaction Diagrams	Show how objects exchange messages to carry out a task. Examples: sequence, communication and timing diagrams.
Communication Diagram	Shows objects as nodes connected by links, with numbered messages that indicate the order of exchange. It was called a collaboration diagram in UML 1.x.

4.5 Linkages Between Concepts

From	Linking Phrase	To	Reason for the Link
UML	is divided into	Structural, Behavioral and Interaction Diagrams	Each group answers a different question about the system.
Structural Diagrams	includes the instance-level view	Object Diagram	Shows the objects that exist at one moment.
Object Diagram	is a snapshot of	Class Diagram	Objects are instances of the classes defined in the class diagram.
Behavioral Diagrams	includes the workflow view	Activity Diagram	Shows the flow of activities and decisions.

From	Linking Phrase	To	Reason for the Link
Interaction Diagrams	includes the object-link view	Communication Diagram	Shows messages along the links between objects.
Communication Diagram	presents the same messages as	Sequence Diagram	Communication emphasises links; sequence emphasises time order.

4.6 Explanation

Object diagrams represent object instances and relationships. Where a class diagram describes the general structure (for example, a Customer class linked to an Order class), an object diagram shows a specific situation, such as the customer "Ravi" linked to order number 101 with actual attribute values. This makes it useful for checking whether the class model can describe real situations.

Communication diagrams show message exchange among objects. The objects are drawn as nodes, the links between them are drawn as lines, and each message is numbered (1, 2, 2.1 and so on) to show its order. They are used when the designer wants to see which objects are connected and which messages travel along each link.

In the UML 2.5 specification, interaction diagrams are formally a specialised kind of behavior diagram (Object Management Group, 2017). The question and Figure 3 show them as a separate branch only to make the three groups easier to compare.

4.7 Analysis

- **Improves system visualization:** a diagram makes both the static structure and the dynamic behaviour visible before any code is written.
- **Supports object-oriented analysis:** object and class diagrams describe the problem domain, while interaction diagrams show how objects collaborate to realise each use case.
- **Facilitates stakeholder communication:** UML is a common notation, so analysts, designers, programmers and clients can discuss the same model.

4.8 Support from Literature

The classification used in this map follows the OMG UML specification (Object Management Group, 2017). Booch, Rumbaugh and Jacobson (2005) describe the object diagram as a view of instances of the elements found in a class diagram, and Fowler (2003) explains that sequence and communication diagrams express similar information with different emphasis, one on time order and the other on the links between objects.

5. Integration of the Three Concept Maps

The three concept maps describe one connected process. The life cycle in Concept Map 1 sets the order of work, the principles in Concept Map 2 guide how the design is structured, and the UML diagrams in Concept Map 3 record that design. The table below shows where each idea is applied in each life-cycle phase.

SDLC Phase	Object-Oriented Principle Applied	UML Diagram Commonly Used
Requirement Analysis	Abstraction: identify the essential domain concepts only.	Use case and activity diagrams
Design	Encapsulation, inheritance and polymorphism: structure the classes and their behaviour.	Class, sequence and communication diagrams
Coding	All four principles are realised in class code.	Class diagram as the reference for implementation
Testing	Polymorphic calls are tested against each subclass.	Object diagrams to describe test scenarios
Deployment	Encapsulated components can be installed and replaced independently.	Component and deployment diagrams
Maintenance	Encapsulation limits how far a change spreads through the system.	Updated class and interaction diagrams

This shows that the concepts in the three maps are not separate topics. Design and Deployment (Concept Map 1) are the phases in which Encapsulation and Polymorphism (Concept Map 2) are applied and in which Object and Communication diagrams (Concept Map 3) are drawn, which is why these missing concepts were important to identify correctly.

6. Conclusion

Three concept maps were constructed and completed: the Software Development Life Cycle (missing concepts Design and Deployment), the Object-Oriented Principles (missing concepts Encapsulation and Polymorphism), and the UML diagram types (missing concepts Object Diagram and Communication Diagram). Each map was organised hierarchically, each link was labelled, and each answer was explained, analysed and supported with standard literature.

Together they show how a structured process, sound object-oriented principles and a standard modelling notation combine to produce structured and modular software systems, which is the aim of Course Outcome CO1.

7. Alignment with the Assessment Rubric

Rubric Criterion	Weightage	Where it is Addressed in this Document
Concept Identification	25%	Sections 2.4, 3.4 and 4.4 define every given, missing and supporting concept.
Linkages	25%	Labelled arrows in Figures 1 to 3 and the linkage tables in Sections 2.5, 3.5 and 4.5.
Organization	20%	Top-down hierarchy in every map, colour coding, and the same section structure for each question.
Integration of Literature	20%	Sections 2.8, 3.8 and 4.8, the integration in Section 5, and the References.
Clarity	10%	Legend and captions for every figure, consistent Times New Roman formatting with 1.5 line spacing.

References

1. Boehm, B. W. (1981). *Software engineering economics*. Prentice Hall.
2. Booch, G., Maksimchuk, R. A., Engel, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3rd ed.). Addison-Wesley.
3. Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2nd ed.). Addison-Wesley.
4. Cardelli, L., & Wegner, P. (1985). On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4), 471–522.
5. Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.
6. Meyer, B. (1997). *Object-oriented software construction* (2nd ed.). Prentice Hall.
7. Novak, J. D., & Cañas, A. J. (2008). *The theory underlying concept maps and how to construct and use them* (Technical Report IHMC CmapTools 2006-01 Rev 01-2008). Florida Institute for Human and Machine Cognition.
8. Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. <https://www.omg.org/spec/UML/2.5.1>
9. Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058.
10. Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
11. Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 1–9.
12. Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

CONCEPT MAPPING

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are wellstructured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand